Theory of Multiclass Classification using Deep Neural Network

OCR Letter Recognition Dataset

This practical implements **Multiclass Classification using Deep Neural Networks (DNN)** to recognize English alphabet letters (A–Z) from image-based features.

The model predicts one class out of 26 possible classes.

---

1. What is Classification?

Classification is a **supervised machine learning technique** used to predict categorical labels.

Examples:

- Spam / Not Spam
- Positive / Negative review
- Disease / No Disease
- Alphabet recognition (A–Z)

---

2. What is Multiclass Classification?

Multiclass classification means:

One input belongs to one class among multiple classes.

In this project:

Input	Output
Letter features	One alphabet (A–Z)

Total classes = 26

---

3. OCR Letter Recognition Dataset

OCR = Optical Character Recognition

The dataset contains:

- 20,000 samples
- 16 numerical features
- 26 output classes (A–Z)

Dataset Structure

Column	Meaning
First column	Target letter
Remaining 16 columns	Image features

Example:

Features	Target
Numerical values	A
Numerical values	B
Numerical values	X

---

4. Importing Libraries

```
import numpy as np
import pandas as pd
```

NumPy

Used for:

- arrays
- mathematical operations

Pandas

Used for:

- reading dataset
- handling tables/dataframes

---

5. Loading Dataset

```
df = pd.read_csv("letter-recognition.data")
```

Reads CSV file into dataframe.

---

6. Feature and Target Separation

```
x = df.iloc[:,1:]
y = df.iloc[:,0]
```

x

Contains input features (16 columns)

y

Contains output labels (A–Z)

---

7. Label Encoding

```
from sklearn.preprocessing import LabelEncoder


encoder = LabelEncoder()
y = encoder.fit_transform(y)
```

Neural networks work with numbers, not text labels.

So letters are converted into integers.

Example:

Letter	Encoded Value
A	0
B	1
C	2
...	...
Z	25

Why Label Encoding is Needed?

Because neural networks perform mathematical computations.

They cannot directly understand:

A, B, C

They need:

0, 1, 2

---

8. Train-Test Split

```
x_train,x_test,y_train,y_test = train_test_split(
    x,y,test_size=0.2,random_state=42)
```

Dataset is divided into:

Data	Percentage
Training Data	80%
Testing Data	20%

Purpose

Training Data

Used for learning patterns.

Testing Data

Used for evaluating model performance on unseen data.

---

9. Feature Scaling (Standardization)

```
scalar = StandardScaler()


x_train = scalar.fit_transform(x_train)
x_test = scalar.fit_transform(x_test)
```

Standardization scales features so all columns have similar range.

Formula:

genui{"math_block_widget_always_prefetch_v2":{"content":"z = \frac{x-\mu}{\sigma}"}}

Where:

Symbol	Meaning
(x)	Original value
(\mu)	Mean
(\sigma)	Standard deviation

Why Scaling is Important?

Without scaling:

- large-value features dominate
- training becomes unstable
- convergence becomes slow

Scaling improves:

- training speed
- accuracy
- stability

---

10. TensorFlow and Keras

import tensorflow as tf

TensorFlow is Google's deep learning framework.

Keras is used to:

- build neural networks easily
- train models
- evaluate predictions

---

11. Sequential Model

model = Sequential()

Sequential model means:

- layers are connected one after another.

---

12. Dense Layers

model.add(Dense(128,activation='relu'))

Dense layer means:

Every neuron is connected to every neuron in next layer.

Network Architecture

Input Layer → Hidden Layer 1 → Hidden Layer 2 → Output Layer

Your model:

Layer	Neurons
Input Layer	16
Hidden Layer 1	128
Hidden Layer 2	64
Output Layer	26

---

13. Activation Function — ReLU

activation='relu'

ReLU = Rectified Linear Unit

Formula:

$f(x)=\max(0,x)$

Meaning:

- Positive input → unchanged
- Negative input → 0

Why ReLU is Used?

Advantages:

- Faster training
- Reduces vanishing gradient problem
- Computationally efficient

---

14. Output Layer with Softmax

model.add(Dense(26,activation='softmax'))

Output layer has:

- 26 neurons → one for each alphabet

Softmax Activation Function

Softmax converts outputs into probabilities.

Formula:

$P(y_i)=\frac{e^{z_i}}{\sum_{j=1}^{n}e^{z_j}}$

Example

Suppose output:

Letter	Score
A	2.1

Letter	Score
B	1.3
C	4.5

Softmax converts them into:

Letter	Probability
A	0.10
B	0.04
C	0.86

Highest probability becomes predicted class.

---

15. Model Compilation

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Optimizer — Adam

Adam optimizer updates weights automatically to minimize loss.

Advantages:

- Fast training
- Adaptive learning rate
- Stable optimization

Loss Function — Sparse Categorical Crossentropy

Used for multiclass classification when labels are integer encoded.

Formula:

$$L = -\log(P(y_{true}))$$

Where:

- $(P(y_{true}))$ = probability of correct class

Why Sparse Categorical Crossentropy?

Because:

- output has multiple classes
- labels are integers (0–25)

If labels were one-hot encoded, we would use:

categorical_crossentropy

Accuracy Metric

metrics=['accuracy']

Accuracy formula:

Accuracy = \frac{Correct\ Predictions}{Total\ Predictions}

---

16. Model Training

```
history = model.fit(
    x_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.1
)
```

Epoch

One complete pass through training data.

You used:

20 epochs

Meaning model learned from data 20 times.

Batch Size

batch_size=32

32 samples processed at once before updating weights.

Validation Split

validation_split=0.1

10% training data used for validation.

Purpose:

- monitor overfitting
- evaluate during training

---

17. Understanding Training Output

Example:

Epoch 1:

accuracy: 0.43

Epoch 20:

accuracy: 0.97

This means:

- model learned patterns successfully
- accuracy improved significantly
- loss decreased steadily

---

18. Model Evaluation

loss,accuracy = model.evaluate(x_test,y_test)

Evaluates performance on unseen test data.

Your result:

Accuracy = 94.5%

Meaning:

Model correctly predicts letters about 94 times out of 100.

---

19. Prediction

prediction = model.predict(x_test)

Model outputs probabilities for all 26 classes.

Example:

[0.0001, 0.0003, ..., 0.9994]

Highest probability corresponds to predicted letter.

---

20. np.argmax()

predicted_class = np.argmax(prediction[0])

Returns index having maximum probability.

Example:

Probability	Index
0.9994	23

Predicted class = 23

---

21. Inverse Transform

letter = encoder.inverse_transform([predicted_class])

Converts encoded number back into original letter.

Example:

Encoded	Letter
23	X

Output:

['X']

---

22. Complete Workflow

Load Dataset

↓

Separate Features & Labels

↓

Encode Labels

↓

Split Training & Testing Data

↓

Scale Features

↓

Build DNN Model

↓

Compile Model

↓

Train Model

↓

Evaluate Accuracy

↓

Predict Letter Class

---

23. Difference Between Binary and Multiclass Classification

Binary Classification	Multiclass Classification
2 classes	More than 2 classes
Yes/No	A–Z
Spam/Not Spam	Letter Recognition
Sigmoid output	Softmax output

---

24. Why Deep Neural Network Works Well Here?

DNN learns:

- complex feature relationships
- hidden image patterns
- nonlinear mappings

This gives:

- high accuracy
- robust classification
- automatic feature learning

---

25. Conclusion

In this practical:

- OCR Letter Recognition dataset was used
- Labels were encoded numerically
- Features were standardized
- Deep Neural Network was built using TensorFlow/Keras
- Softmax used for multiclass prediction
- Sparse categorical crossentropy used as loss function
- Model achieved approximately 94–95% accuracy

This practical demonstrates how Deep Learning can effectively solve multiclass classification problems.